

`if-elif-else` Statements

`if` Statements

In Python `if` statement evaluates a condition (like `3 > 0` in the following example) then runs a block of code only if the condition is true.

In [29]:
```
if 3 > 0:
    print("The number 3 is positive")
```

```
The number 3 is positive
```

since `3 > 0` is true then Python evaluated the code: `print('The number 3 is positive')`

On the other hand

In [30]:
```
if 3 < 0:
    print("The number 3 is negative")
```

since `3 < 0` is false then Python skiped the code: `print('The number 3 is positive')`. So there is no output.

Indentation

Warning: In Python indentation (space at the beginning of the line) is important.

- New line after `:` must be indented. This is called a **code block**.
- If there is more than one statement inside an `if` block they must be indented exactly like the first one.

In [31]:
```
if 3 == 3.0:
    # We are testing equality with == operator not = assignment
    msg = "The number 3 is equal to 3.0"
    print(msg)
```

```
The number 3 is equal to 3.0
```

But there is an exception to the indentation rule: If there is an open paranthesis then the indentation rules are not required until closing the paranthesis.

In [32]:
```
if 3 >= 3:
    print(
"3 is greater than or equal to 3"

            )
```

```
3 is greater than or equal to 3
```

If there is only one statement in the `if` block then it can be written after `:` without new line and indentation.

In [33]:
```
if 3 != 4: print("3 is not equal to 4")
```

```
3 is not equal to 4
```

`else` Statement

`else` statement runs a block of code only if the condition of the preceeding `if` statement was evaluated false.

In [34]:
```
if 3 == 0:
    print("The number 3 is zero")

else:
    print("The number 3 is not zero")
```

```
The number 3 is not zero
```

`else` statement must be in the same indentaion level with its `if` statement.

Nested `if` statements

In [35]:
```
if 3 < 0 and 3 % 2 == 1:
  print("the number 3 is negative odd onumber")
else:
  print("the number 3 is not negative odd number")
```

```
the number 3 is not negative odd number
```

Additional conditions can be put by nesting multiple `if` statements.

So the previous code can be written by nested `if` statements:

In [36]:
```
if 3 > 0:
    if 3 % 2 == 1:
        print("The number 3 is a positive odd number")
    else:
        print("The number 3 is a positive even number")
else:
    print("The number 3 is not positive")
```

```
The number 3 is a positive odd number
```

Warning: After each nested `if` statement the indentation must be increased.

In [37]:
```
if 3 < 0:
    print("The number 3 is negative")
else:
    if 3 % 2 == 0:
        print("The number 3 is a negative even number")
    else:
        print("The number 3 is a negative odd number")
```

```
The number 3 is a negative odd number
```

`elif` statement

Instead of `else: if` statement use `elif` statement. The advantage of using `elif` is that, the indentation level is same with its `if` statement. This makes code more readable.

In [38]:
```
if 3 < 0:
    print("The number 3 is negative")
elif 3 % 2 == 0:
  print("The number 3 is a non negative even number")
else:
    print("The number 3 is a non negative odd number")
```

```
The number 3 is a non negative odd number
```

`match-case` Statements

In `if-elif-else` statements if the conditions are equality check with a fixed object `match-case` statement are more handy.

In each `case` statement the equality is tested for each `|` seperated objects. If none of the equalities hold then that `case _:` block is executed.

In [39]:
```
day = "sunday"

match day:
    case "monday" | "tuesday" | "wednesday" | "thursday" | "friday":
        print("It's a weekday")
    case "saturday" | "sunday":
        print("It's a weekend")
    case _:
        print("Not a valid day")
```

```
It's a weekend
```

Warning: Here;

- `|` is not the bitwise or operator.
- `_` is not a variable.

`if-else` Expression

Instead of using `if-else` statements in Python, `if-else` expressions are usually more usefull.

In [40]:
```
n = 78

print(n, "is", "even" if n % 2 == 0 else "odd")
```

```
78 is even
```

is equivalent to:

In [41]:
```
n = 7

if n % 2 == 0:
    print(n, "is even")
```

```
else:
    print(n, "is odd")
```

```
7 is odd
```

`if` - `else` expressions can be nested as well:

In [42]:
```
print(n, "is",
      "positive" if n > 0 else
      "negative" if n < 0 else
      "zero")
```

```
7 is positive
```

This is equivalent to:

In [43]:
```
if n > 0:
    print(n, "is positive")
elif n < 0:
    print(n, "is negative")
else:
    print(n, "is zero")
```

```
7 is positive
```

Expressions are more useful because they return an object so it can be assigned to a variable or passed as an argument to a function.

Warning: the `else` part of `if` - `else` expressions can not be ommited unlike `if` - `else` statements.

Truth Value of Python Objects

Every object in Python has a truth value. Builtin `bool()` function can be used to get the truth value of an object.

Objects relateded to voidness are `False` :

In [44]:
```
bool(None), bool(0), bool(""), bool(()), bool([]), bool(set()), bool({})
```

Out[44]:
```
(False, False, False, False, False, False, False)
```

All other objects are `True` :

In [45]:
```
bool(...), bool(-1), bool("Hello"), bool((1,)), bool([0]), bool({"a"}), bool({"a": 1})
```

Out[45]:
```
(True, True, True, True, True, True, True)
```

Therefore, any Python object can be used as a condition in `if` statements and `if` - `else` expressions.

In [46]:
```
print(n, "is", "odd" if n % 2 else "even")
```

```
7 is odd
```

`all()` & `any()` Functions

Builtin `all()` and `any()` functions can be used to test the truth of the items of an iterable.

In [47]:
```
all((1 == 1., 1 < 2, 4 > 3, 3 != 4))
```

Out[47]: True

In [51]:
```
all((..., -1, "Hello", (1,), [0], {"a"}, {"a": 1}))
```

Out[51]: True

In [48]:
```
all((0, 1, 2, 3))
```

Out[48]: False

In [50]:
```
any((1 == 1., 1 > 2, 4 < 3, 3 == 4))
```

Out[50]: True

In [52]:
```
any((None, 0, "", (), [], set(), {}))
```

Out[52]: False

`and` & `or` Expressions

Every Python object has a truth value and every Python expression returns an object. This can be used to control flow of the execution by `and` & `or` expressions.

`and`

In `expr1 and expr2`, Python first evaluates the `expr1`. If the result of the `expr1` is equal to `True` then evaluates and gets the value of `expr2`. Otherwise it gets `expr1`. Mathematically;

$$p \text{ and } q = \begin{cases} p & \text{, if the truth value of } p \text{ is false} \\ q & \text{, if the truth value of } p \text{ is true} \end{cases}$$

In [53]:
```
name    = input('Enter your name: ')
surname = name and input('Enter your surname: ')
```

In this example, the surname will be asked only if the name entered.

`or`

In `expr1 or expr2`, Pyton first evaluates the `expr1`. If the result of the `expr1` is equal to `False`, then evaluates and gets the value of `expr2`. Otherwise it gets `expr1`. Mathematically;

$$p \text{ or } q = \begin{cases} p & \text{, if the truth value of } p \text{ is true} \\ q & \text{, if the truth value of } p \text{ is false} \end{cases}$$

In [54]:
```
name = input('enter your name: ') or input('please enter your name: ')
```